International Islamic University Chittagong
Department of Computer Science & Engineering
Autumn - 2022

Course Code: CSE-2321

Course Title: Data Structures

Mohammed Shamsul Alam
Professor, Dept. of CSE, IIUC

Lecture – 8

Recursion

Recursion

- A function that **calls itself** is termed as a ***recursive function*** and the process involved in doing this is called **recursion**.
- Recursion is an essential concept in Computer Science.
- Recursion makes it convenient to solve various problems that would be cumbersome to solve using iterative constructs such as for, while, and do while.
- Recursion is a methodology that permits breaking down of a problem into one or many sub problems that are similar to the original problem

a function is said to be *recursively defined* if the function definition refers to itself. Again, in order for the definition not to be circular, it must have the following two properties:

(1) There must be certain arguments, called *base values*, for which the function does not refer to itself.

(2) Each time the function does refer to itself, the argument of the function must be closer to a base value

A recursive function with these two properties is also said to be well-defined.

Factorial Function

- A classic example of recursion is the definition of the **factorial** function.

Definition 6.1: (Factorial Function)

(a) If $n = 0$, then $n! = 1$.
(b) If $n > 0$, then $n! = n \cdot (n - 1)!$

Observe that this definition of $n!$ is recursive, since it refers to itself when it uses $(n - 1)!$ However, (a) the value of $n!$ is explicitly given when $n = 0$ (thus 0 is the base value); and (b) the value of $n!$ for arbitrary n is defined in terms of a smaller value of n which is closer to the base value 0. Accordingly, the definition is not circular, or in other words, the procedure is well-defined.

(1) $4! = 4 \cdot 3!$
(2) $3! = 3 \cdot 2!$
(3) $2! = 2 \cdot 1!$
(4) $1! = 1 \cdot 0!$
(5) $0! = 1$
(6) $1! = 1 \cdot 1 = 1$
(7) $2! = 2 \cdot 1 = 2$
(8) $3! = 3 \cdot 2 = 6$
(9) $4! = 4 \cdot 6 = 24$

Factorial Function

The following are two procedures that each calculate *n* factorial.

Procedure 6.9A: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Set FACT := 1 [Initializes FACT for loop.]
3. Repeat for K = 1 to N.
   Set FACT := K*FACT.
   [End of loop.]
4. Return.

Procedure 6.9B: FACTORIAL(FACT, N)

This procedure calculates N! and returns the value in the variable FACT.

1. If N = 0, then: Set FACT := 1, and Return.
2. Call FACTORIAL(FACT, N – 1).
3. Set FACT := N*FACT.
4. Return.

Fibonacci Sequence

The celebrated Fibonacci sequence (usually denoted by $F_0, F_1, F_2, \ldots$) is as follows:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots.$$

That is, $F_0 = 0$ and $F_1 = 1$ and each succeeding term is the sum of the two preceding terms. For example, the next two terms of the sequence are

$$34 + 55 = 89 \quad \text{and} \quad 55 + 89 = 144$$

A formal definition of this function follows:

Definition 6.2: (Fibonacci Sequence)

(a) If $n = 0$ or $n = 1$, then $F_n = n$.
(b) If $n > 1$, then $F_n = F_{n-2} + F_{n-1}$.

This is another example of a recursive definition, since the definition refers to itself when it uses F_{n-2} and F_{n-1}. Here (a) the base values are 0 and 1, and (b) the value of F_n is defined in terms of smaller values of n which are closer to the base values. Accordingly, this function is well-defined.

Fibonacci Sequence

FIBONACCI(F, N)
This procedure finds the first N Fibonacci numbers and assigns them to an array F.

1. Set F[1] := 1 and F[2] := 1.
2. Repeat for L = 3 to N:
   Set F[L] := F[L – 1] + F[L – 2].
   [End of loop.]
3. Return.

A procedure for finding the nth term F_n of the Fibonacci sequence follows.

Procedure 6.10: FIBONACCI(FIB, N)
This procedure calculates F_N and returns the value in the first parameter FIB.

1. If N = 0 or N = 1, then: Set FIB := N, and Return.
2. Call FIBONACCI(FIBA, N – 2).
3. Call FIBONACCI(FIBB, N – 1).
4. Set FIB := FIBA + FIBB.
5. Return.

This is another example of a recursive procedure, since the procedure contains a call to itself.

Divide & Conquer Algorithms

Consider a problem P associated with a set S. Suppose A is an algorithm which partitions S into smaller sets such that the solution of the problem P for S is reduced to the solution of P for one or more of the smaller sets. Then A is called a divide-and-conquer algorithm.

Ackermann Function

The Ackermann function is a function with two arguments each of which can be assigned any nonnegative integer: 0, 1, 2,.... This function is defined as follows:

Definition 6.3: (Ackermann Function)

(a) If $m = 0$, then $A(m, n) = n + 1$.
(b) If $m \neq 0$ but $n = 0$, then $A(m, n) = A(m - 1, 1)$.
(c) If $m \neq 0$ and $n \neq 0$, then $A(m, n) = A(m - 1, A(m, n - 1))$

Find the value of A(1,3).

Ackermann Function

Find the value of A(1,3).

(1) $A(1, 3) = A(0, A(1, 2))$
(2) $A(1, 2) = A(0, A(1, 1))$
(3) $A(1, 1) = A(0, A(1, 0))$
(4) $A(1, 0) = A(0, 1)$
(5) $A(0, 1) = 1 + 1 = 2$
(6) $A(1, 0) = 2$
(7) $A(1, 1) = A(0, 2)$
(8) $A(0, 2) = 2 + 1 = 3$
(9) $A(1, 1) = 3$
(10) $A(1, 2) = A(0, 3)$
(11) $A(0, 3) = 3 + 1 = 4$
(12) $A(1, 2) = 4$
(13) $A(1, 3) = A(0, 4)$
(14) $A(0, 4) = 4 + 1 = 5$
(15) $A(1, 3) = 5$

Towers of Hanoi

Suppose three pegs, labeled A, B and C, are given, and suppose on peg A there are placed a finite number n of disks with decreasing size. This is pictured in Fig. 6.14 for the case $n = 6$. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary. The rules of the game are as follows:

(a) Only one disk may be moved at a time. Specifically, only the top disk on any peg may be moved to any other peg.

(b) At no time can a larger disk be placed on a smaller disk.

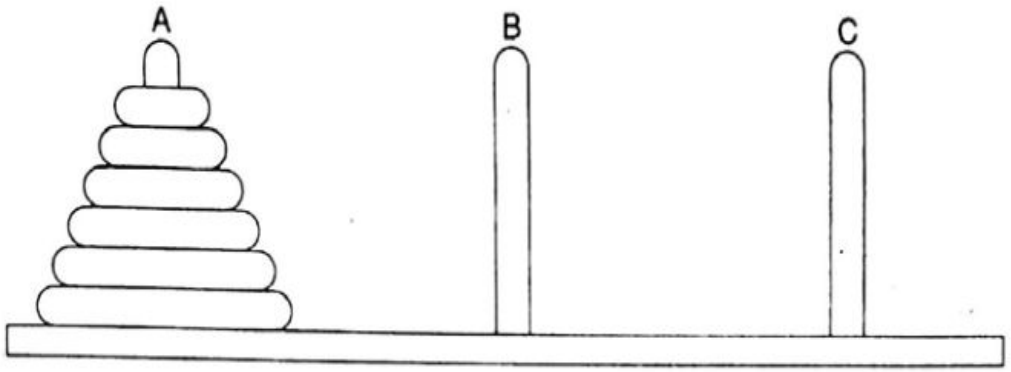

Fig. 6.14 *Initial Setup of Towers of Hanoi with n = 6*

Towers of Hanoi

TOWER(N, BEG, AUX, END)
This procedure gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If N = 1, then:
   (a) Write: BEG → END.
   (b) Return.
   [End of If structure.]
2. [Move N – 1 disks from peg BEG to peg AUX.]
   Call TOWER(N – 1, BEG, END, AUX).
3. Write: BEG → END.
4. [Move N – 1 disks from peg AUX to peg END.]
   Call TOWER(N – 1, AUX, BEG, END).
5. Return.

Towers of Hanoi

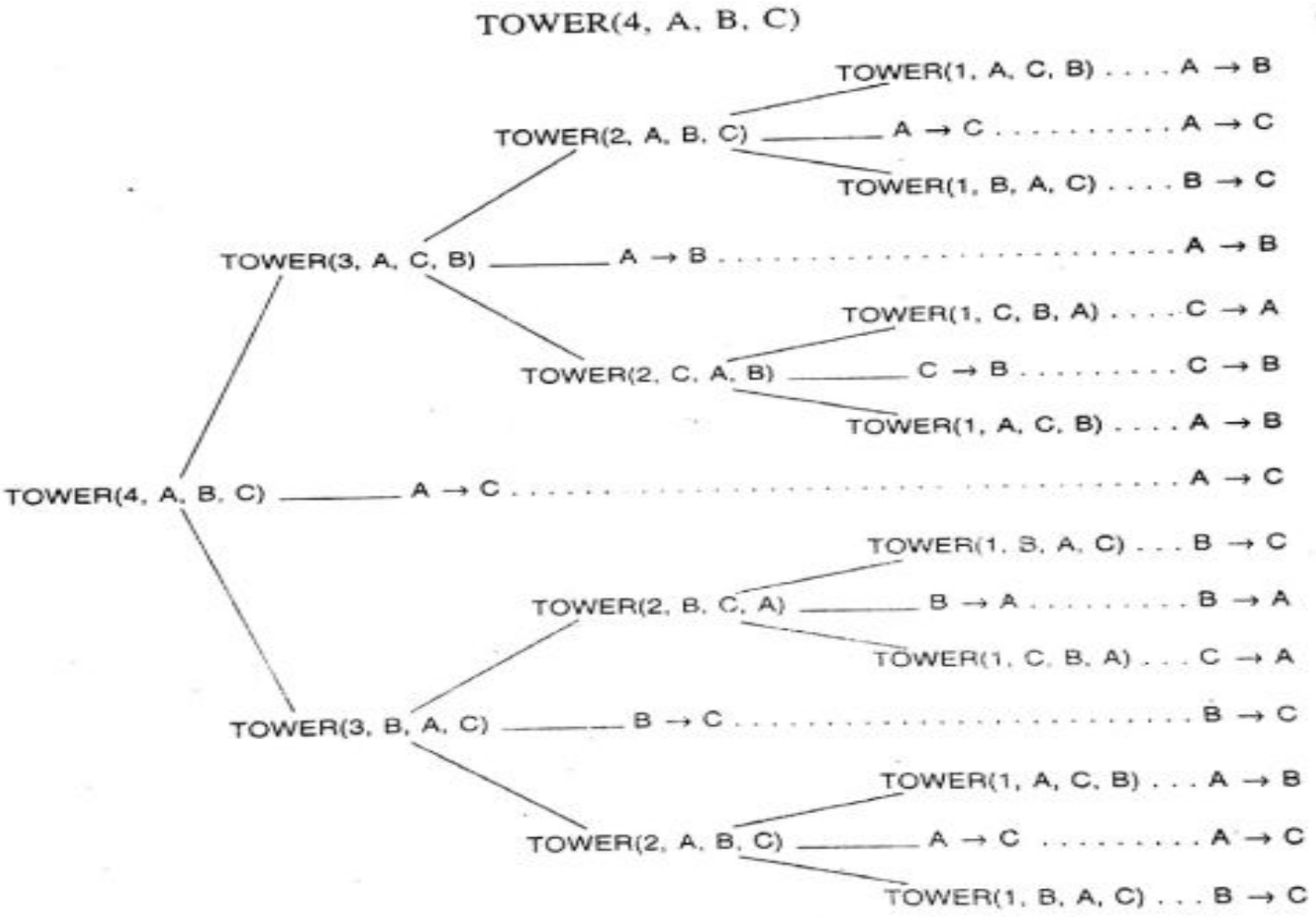

Fig. 6.17 *Recursive Solution to Towers of Hanoi Problem for n = 4*

Observe that the recursive solution for $n = 4$ disks consists of the following 15 moves:

$A \to B \quad A \to C \quad B \to C \quad A \to B \quad C \to A \quad C \to B \quad A \to B \quad A \to C$
$B \to C \quad B \to A \quad C \to A \quad B \to C \quad A \to B \quad A \to C \quad B \to C$

In general, this recursive solution requires $f(n) = 2^n - 1$ moves for n disks.